

Exercises 02

Storage: Disks, Files, Buffers

[\[Show with no answers\]](#) [\[Show with all answers\]](#)

1. What is the purpose of the storage management subsystem of a DBMS?

[\[show answer\]](#)

2. Describe some of the typical functions provided by the storage management subsystem.

[\[show answer\]](#)

3. [Based on Garcia-Molina/Ullman/Widom 13.6.1]

Consider a disk with the following characteristics:

- 8 platters, 16 read/write surfaces
- 16,384 (2^{14}) tracks per surface
- On average, 128 sectors/blocks per track (min: 96, max: 160)
- 4096 (2^{12}) bytes per sector/block

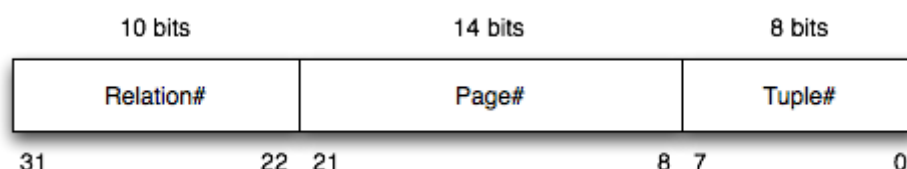
If we represent record addresses on such a disk by allocating a separate byte (or bytes) to address the surface, the track, the sector/block, and the byte-offset within the block, how many bytes do we need?

How would the answer differ if we used bit-fields, used the minimum number of bits for each address component, and packed the components as tightly as possible?

[\[show answer\]](#)

4. The raw disk addresses in the first question are very low level. DBMSs normally deal with higher-level objects than raw disk blocks, and thus use different kinds of addresses, such as **PageIds** and **TupleIds**.

Consider a DBMS where **TupleIds** are defined as 32-bit quantities consisting the following:



Write C functions to extract the various components from a **TupleId** value:

```
typedef unsigned int BitString;
typedef BitString TupleId;

BitString relNum(Tuple id) { ... }
BitString pageNumFrom(Tuple id) { ... }
BitString recNumFrom(Tuple id) { ... }
```

[\[show answer\]](#)

5. Consider executing a nested-loop join on two small tables (R , with $b_R=4$, and S , with $b_S=3$) and using a small buffer pool (with 3 initially unused buffers). The pattern of access to pages is determined by the following algorithm:

```
for (i = 0; i < bR; i++) {
    rpage = request_page(R, i);
    for (j = 0; j < bS; j++) {
        spage = request_page(S, j);
        process join using tuples in rpage and spage ...
        release_page(S, j);
    }
    release_page(R, i);
}
```

Show the state of the buffer pool and any auxiliary data structures after the completion of each call to the **request** or **release** functions. For each buffer slot, show the page that it currently holds and its pin count, using the notation e.g. **R0(1)** to indicate that page 0 from table **R** is held in that buffer slot and has a pin count of 1. Assume that free slots are always used in preference to slots that already contain data, even if the slot with data has a pin count of zero.

In the traces below, we have not explicitly showed the initial free-list of buffers. We assume that **Buf[0]** is at the start of the list, then **Buf[1]**, then **Buf[2]**. The allocation method works as follows, for all replacement strategies:

- if the free-list has any buffers, use the first one on the list
- if the free-list is empty, apply the replacement strategy

The trace below shows the first part of the buffer usage for the above join, using PostgreSQL's clock-sweep replacement strategy. Indicate each read-from-disk operation by a * in the **R** column. Complete this example, and then repeat this exercise for the LRU and MRU buffer replacement strategies.

Operation	Buf[0]	Buf[1]	Buf[2]	R	Strategy data	Notes
initially	free	free	free		NextVictim=0	



request(R0)	R0(1)	free	free	*	NextVictim=0	use first av
request(S0)	R0(1)	S0(1)	free	*	NextVictim=0	use first av
release(S0)	R0(1)	S0(0)	free		NextVictim=0	
request(S1)	R0(1)	S0(0)	S1(1)	*	NextVictim=0	use first av
release(S1)	R0(1)	S0(0)	S1(0)		NextVictim=0	
request(S2)	R0(1)	S2(1)	S1(0)	*	NextVictim=2	skip pinned
release(S2)	R0(1)	S2(0)	S1(0)		NextVictim=2	
release(R0)	R0(0)	S2(0)	S1(0)		NextVictim=2	
request(R1)	R0(0)	S2(0)	R1(1)	*	NextVictim=0	use NextVict
request(S0)	...					
etc. etc. etc.						
release(S2)	...					
release(R3)	...					

[\[show answer\]](#)

6. [Based on GUW Ex.15.7.1]

Consider executing a join operation on two tables R and S . A pool of N buffers is available to assist with the execution of the join. In terms of N , b_R and b_S , give the conditions under which we can guarantee that the tables can be joined in a single pass (i.e. each page of each table is read exactly once). Assume that the join here results in writing result tuples, unlike the previous question, so you need one output buffer as well as input buffers.

[\[show answer\]](#)

7. Consider the execution of a binary search on the sort key in a file where $b=100$. Assume that the key being sought has a value in the middle of the range of values in the data page with index 52. Assume also that we have a buffer pool containing only 2 pages both of which are initially unused. Show the sequence of reads and replacements in the buffer pool during the search, for each of the following page replacement strategies:

- first-in-first-out
- most-recently-used

Use the following notation for describing the sequence of buffer pool operations, e.g.

```
request for page 3
placed in buffer 0
request for page 9
placed in buffer 1
request for page 14
placed in buffer 0 (page 3 replaced)
request for page 19
placed in buffer 1 (page 9 replaced)
etc. etc. etc.
```



Assuming that this is the only process active in the system, does the buffering achieve any disk i/o savings in either case?

[\[show answer\]](#)

8. A commonly used buffer replacement policy in DBMSs is LRU (least recently used).

However, this strategy is not optimal for some kinds of database operations. One proposal to improve the performance of LRU was LRU/ k , which involved using the k^{th} most recent access time as the basis for determining which page to replace. This approach had its own problems, in that it was more complex to manage the buffer queue ($\log N$ time, rather than constant time). The effect of the most popular variant of LRU/ k , LRU/2, is to better estimate how “hot” is a page (based on more than just its most recent, and possibly only, access); pages which are accessed only once recently are more likely to be removed than pages that have been accessed several times, but perhaps not as recently.

PostgreSQL 8.0 and 8.1 used a buffer replacement strategy based on a different approach, called 2Q. The approach uses two queues of buffer pages: A1 and Am. When a page is first accessed, it is placed in the A1 queue. If it is subsequently accessed, it is moved to the Am queue. The A1 queue is organised as a FIFO list, so that pages that are accessed only once are eventually removed. The Am queue is managed as an LRU list. A simple algorithm for 2Q is given below:

```
Request for page p:

if (page p is in the Am queue) {
    move p to the front (LRU) position in the Am queue
}
else if (page p is in the A1 queue) {
    move p to the front (LRU) position in the Am queue
}
else {
    if (there are available free buffers) {
        B = select a free buffer
    }
    else if (size of A1 > 2) {
        B = buffer at head of A1
        remove B from A1
    }
    else {
        B = LRU buffer in Am
        remove B from Am
    }
    allocate p to buffer B
    move p to the tail of the A1 queue (FIFO)
}
```



Using the above algorithm, show the state of the two queues after each of the following page references:

1	2	1	3	1	2	1	4	2	5	6	4	3	5
---	---	---	---	---	---	---	---	---	---	---	---	---	---

To get you started, after the first four references above, the queues will contain:

A1: 2 3	Am: 1	2 free buffers
---------	-------	----------------

Assume that the buffer pool contains 5 buffers, and that it is initially empty. Note that the page nearest to A1 is the head of the FIFO queue (i.e. the next one to be removed according to FIFO), and the page nearest to Am is the least recently used page in that queue.

Note that PostgreSQL changed to the clock-sweep replacement strategy in later releases.

[\[show answer\]](#)

9. Challenge Problem: (no solution provided)

Write a program that simulates the behaviour of a buffer pool. It should take as command-line arguments:

- the number of buffers (an integer value, larger than 2)
- the replacement strategy (one of `clock`, `lru`, `mru`)

It should then read from standard input a sequence of page references, one per line, in the form:

- `req T n`
- `rel T n`

where `T` is a table name and `n` is a page number

The output of the program should be a trace of buffer states in a format similar to that used in Question 8. Also, collect statistics on numbers of requests, releases, reads and writes and display these at the end of the trace.

Since generating long and meaningful sequences of requests and releases is tedious, you should also write programs to generate such sequences. The pseudo-code in Question 8 gives an idea of what the core of such a program would look like for a join on two tables.

